COLLEGE PROJECT DOCUMENTATION

# RAG Assistant: A Retrieval-Augmented Generation Document Assistant

## Project Estimation using the COCOMO Model

| | |
|---|---|
| Document | Project Estimation using the COCOMO Model |
| Version | 1.0 |
| Date prepared | 12 October 2026 |
| Prepared by | |
| Course / Supervisor | |

Planning document. The project is estimated and planned from Step 0: no development is assumed to be complete.

# Contents

1. Introduction
2. Project Scope and Assumptions
3. COCOMO Model Explanation
4. Software Size Estimate
5. COCOMO Calculation
6. Cost Estimation
7. Estimation Summary
8. Limitations and Risks

> All values are estimates prepared before development (Step 0). They are model outputs, not measurements. The companion document *Project Plan using Agile Methodology* schedules the same scope into sprints; Section 5.8 explains how the two relate.

# 1. Introduction

## 1.1 Project title

RAG Assistant: A Retrieval-Augmented Generation Document Assistant

## 1.2 Problem statement

Finding an answer inside long PDFs, notes and reports is slow. Keyword search misses passages that use different wording, and general chatbots can produce answers that are not supported by the user's own documents. Users need an assistant that answers questions from their documents only and shows where each answer came from.

## 1.3 Project objectives

1. Let users securely upload and manage PDF, TXT and Markdown documents.
2. Parse, chunk, embed and index document text for semantic and keyword search.
3. Generate context-grounded answers with an LLM and attach source citations.
4. Keep conversation history and isolate each user's data.
5. Deliver tested, documented software and a final demonstration.

## 1.4 Project scope

A web application (Next.js frontend, FastAPI backend, PostgreSQL, Qdrant) covering authentication, document ingestion, hybrid retrieval with reranking, LLM answer generation with citations, conversation history, document management, testing and documentation. Section 2 lists exactly what is included and excluded.

## 1.5 Purpose of software effort estimation

Effort estimation predicts how much work a project needs before it starts. It supports realistic scheduling, scope decisions, cost planning and risk management. This document uses the Constructive Cost Model (COCOMO), a well-known algorithmic model that derives effort and duration from the estimated size of the software.

# 2. Project Scope and Assumptions

## 2.1 Functional requirements

| ID | Requirement |
|---|---|
| FR1 | User authentication (registration, login, JWT) and authorization (per-user data isolation) |
| FR2 | Upload of PDF, TXT and Markdown documents with validation |
| FR3 | Document parsing and text chunking with page tracking |
| FR4 | Embedding generation (BAAI/bge-small-en-v1.5) and vector indexing in Qdrant |
| FR5 | Semantic (dense) and keyword (BM25) retrieval with Reciprocal Rank Fusion and cross-encoder reranking |
| FR6 | Context-grounded question answering with an LLM (OpenAI API or compatible provider) |
| FR7 | Source citations for every answer |
| FR8 | Conversation history and document management (list, delete) |

## 2.2 Non-functional requirements

| ID | Requirement |
|---|---|
| NFR1 | Security: passwords stored only as salted hashes; token-based authentication; every query restricted to the requesting user's data |
| NFR2 | Reliability: invalid input and service failures return clear errors; failed uploads leave no partial data |
| NFR3 | Responsiveness: heavy parsing and embedding do not block other requests; upload size limited (about 50 MB) |
| NFR4 | Grounding: answers use retrieved document content only and cite their sources |
| NFR5 | Maintainability and testability: modular code with automated unit and API tests |
| NFR6 | Usability: clear upload progress, loading and error states in the web interface |

## 2.3 Planned technology stack

| Area | Technology |
|---|---|
| Frontend | Next.js, TypeScript, Tailwind CSS |
| Backend | Python, FastAPI, Pydantic |
| Relational database | PostgreSQL |
| Vector database | Qdrant |
| Embedding model | BAAI/bge-small-en-v1.5 (sentence-transformers, runs locally) |
| Retrieval | Dense vector search and BM25 hybrid retrieval |
| Ranking | Reciprocal Rank Fusion (RRF) and cross-encoder reranking |
| LLM | OpenAI API or a compatible provider |
| Testing | pytest (backend); Node test runner, TypeScript and ESLint checks (frontend) |

Verified against the repository's dependency files (backend requirements and frontend package manifest) and Docker Compose file.

## 2.4 Estimation assumptions

- **Team:** one student developer with working knowledge of Python, TypeScript, SQL and Git; new to some RAG components (vector search, reranking).
- **Working hours:** about 3 focused hours per working day, 5 working days per week.
- **Start:** the project starts at Step 0. Any code already in the repository is not counted as completed work.
- **Reuse:** open-source libraries and pretrained models are used, not written. Their code is not counted in KLOC.
- **Person-month:** 152 working hours, the standard value used with COCOMO.

## 2.5 Included and excluded from the estimate

| Included | Excluded |
|---|---|
| All functional requirements FR1-FR8; NFR1-NFR6; error handling; automated tests; documentation and demonstration | OCR for scanned PDFs; other file formats (DOCX, PPTX, images); streaming answers; document sharing; admin dashboard; model training or fine-tuning; production-scale hosting. Public deployment is optional and costed separately. |

# 3. COCOMO Model Explanation

## 3.1 What COCOMO is

The Constructive Cost Model (COCOMO) was published by Barry Boehm in 1981. It estimates software development effort and schedule from the expected size of the delivered source code, measured in thousands of lines of code (KLOC). COCOMO has three levels: **Basic** (size only), **Intermediate** (size plus 15 cost drivers) and **Detailed** (cost drivers applied per phase).

**Selected model: Basic COCOMO.** At Step 0 there is not enough reliable information to rate the 15 Intermediate cost drivers (for example product complexity or analyst capability) objectively, so the simpler Basic model is the most honest choice for an early estimate.

## 3.2 Equations

$$E = a \times (KLOC)^{b}$$

$$T = c \times (E)^{d}$$

$$P = E / T$$

| Symbol | Meaning | Unit |
|---|---|---|
| E | Estimated development effort | person-months |
| T | Estimated development time (schedule) | months |
| P | Average staffing over the schedule | persons |
| KLOC | Estimated delivered source code size | thousands of lines of code |
| a, b | Effort coefficients for the selected project mode | - |
| c, d | Schedule coefficients for the selected project mode | - |

## 3.3 Project modes and coefficients (Basic COCOMO, Boehm 1981)

| Mode | Typical project | a | b | c | d |
|---|---|---|---|---|---|
| Organic | Small team, familiar problem and environment, flexible requirements, typically under 50 KLOC | 2.4 | 1.05 | 2.5 | 0.38 |
| Semi-detached | Medium size, mixed experience, some rigid requirements or unfamiliar parts | 3.0 | 1.12 | 2.5 | 0.35 |
| Embedded | Tight hardware, software or regulatory constraints; complex, rigid requirements | 3.6 | 1.20 | 2.5 | 0.32 |

## 3.4 Mode selection and justification

**Selected mode: Organic.**

- The system is small (about 5 KLOC, far below 50 KLOC) and built by one developer.

- Requirements are flexible: features can be reprioritised during development without contractual or regulatory constraints.
- There are no hardware, real-time or safety constraints; it is a standard web application.
- The main novelty (retrieval and LLM integration) is handled through mature libraries and a managed API rather than new low-level algorithms.

Semi-detached was considered because RAG components are unfamiliar to a student. It was not selected because the size, team and constraint characteristics are clearly organic; its result is still shown in Section 5.9 for transparency. The mode was chosen from project characteristics, not to obtain a particular result.

## 3.5 Limitations of Basic COCOMO

- It depends entirely on a KLOC estimate, which is uncertain early in a project.
- It was calibrated on 1970s industrial projects using older languages and full documentation-heavy life cycles.
- It ignores cost drivers such as team experience, tools and code reuse.
- Lines of code vary by language and coding style, and do not capture work hidden in libraries, models and configuration.

# 4. Software Size Estimate

## 4.1 Size by module

Size is estimated as non-blank, non-comment lines of hand-written source code that will be delivered. Each module was sized by listing its planned components and estimating typical lines for each (decomposition and expert judgement), with a low-high range to show uncertainty.

| Module | Planned contents | Low | Nominal | High | Backlog items |
|---|---|---|---|---|---|
| M1 Frontend web application | Next.js pages, React components, hooks, API client, TypeScript types, styles | 1.6 | 2.1 | 2.6 | PB-18 to PB-21 |
| M2 Backend API layer | FastAPI app setup, routers, request/response schemas, validation, configuration | 0.6 | 0.8 | 1.0 | PB-01, PB-02, PB-22 |
| M3 Document ingestion | Upload handling, PDF/TXT/MD parsers, chunking, ingestion pipeline and cleanup | 0.3 | 0.4 | 0.6 | PB-05 to PB-07 |
| M4 Embeddings and vector store | Embedding wrapper (bge-small-en-v1.5), Qdrant collection, upsert, filtered search, delete | 0.2 | 0.3 | 0.4 | PB-08, PB-09 |
| M5 Retrieval and ranking | Dense retrieval, BM25 keyword search, Reciprocal Rank Fusion, cross-encoder reranking | 0.2 | 0.3 | 0.4 | PB-11, PB-16, PB-17 |
| M6 RAG generation and citations | Prompt construction, LLM client, context building, citations, follow-up rewriting | 0.2 | 0.3 | 0.4 | PB-12, PB-13, PB-15 |
| M7 Authentication, history and document management | User accounts, password hashing, JWT, user isolation, PostgreSQL schema and stores, conversations | 0.6 | 0.8 | 1.0 | PB-03, PB-04, PB-10, PB-14 |
| **Total delivered application code (KLOC)** | | **3.7** | **5.0** | **6.4** | |

## 4.2 What is and is not counted

| Counted in KLOC | Not counted in KLOC |
|---|---|
| Hand-written backend and frontend source code that ships with the application (Python, TypeScript/TSX, application CSS) | Automated test code (about 2.0 KLOC planned; its effort is covered by COCOMO's life-cycle effort); third-party libraries and frameworks; pretrained model files (embedding model, reranker); generated code and build output; lock files; configuration (.env, Docker Compose); documentation |

## 4.3 Plausibility check

The project repository already contains an early implementation. Measured only as a reference for typical module sizes, and **not** counted as completed work, it has roughly 2.3 KLOC of backend application code and 2.2 KLOC of frontend code (non-blank lines). This is consistent with the nominal 5.0 KLOC estimate.

> The size estimate is preliminary (rounded to 0.1 KLOC; an uncertainty of about ±30% is normal at this stage). It will change as requirements become more detailed and should be re-estimated after the first sprint.

# 5. COCOMO Calculation

**Step 1: Project mode**
Organic (justified in Section 3.4).

**Step 2: Coefficients (Basic COCOMO, organic)**

$$a = 2.4,\ \ b = 1.05,\ \ c = 2.5,\ \ d = 0.38$$

**Step 3: Substitute KLOC into the effort equation**

$$E = a \times (KLOC)^{b} = 2.4 \times (5.0)^{1.05}$$

$$(5.0)^{1.05} = 5.4190$$

**Step 4: Effort**

$$E = 2.4 \times 5.4190 = \mathbf{13.01\ person\text{-}months}$$

**Step 5: Development time**

$$T = c \times (E)^{d} = 2.5 \times (13.01)^{0.38}$$

$$(13.01)^{0.38} = 2.6508$$

$$T = 2.5 \times 2.6508 = \mathbf{6.63\ months}$$

**Step 6: Average staffing**

$$P = E / T = 13.01 / 6.63 = \mathbf{1.96\ persons}$$

**Step 7: Convert effort to person-hours (152 hours per person-month)**

$$\text{Person-hours} = E \times 152 = 13.01 \times 152 \approx \mathbf{1{,}977\ hours}$$

## 5.8 Step 8: What this means for the student schedule

- COCOMO's model answer is about **13 person-months** over **6.6 months** with about **2 people**.
- For a single full-time developer (P = 1), the duration would be about E / 1 ≈ 13 months.
- At the planned student pace of 3 hours per working day, 1,977 hours would take about 659 working days, which is far longer than an assignment allows.
- The Agile plan therefore does not schedule the COCOMO effort directly. It fixes the time box (4 two-week sprints, about 120 hours of student capacity) and manages scope by priority: Must-have items first, Should-have and Could-have items only if capacity remains.

**Why the gap exists:** Basic COCOMO reflects industrial projects with full specification, design reviews, formal testing and documentation, written largely without today's frameworks. This project relies heavily on frameworks (Next.js, FastAPI), managed services (LLM API) and pretrained models, which COCOMO's

size-only model does not credit. The gap is not hidden: it is treated as a key schedule risk (Section 8), and progress will be measured after Sprint 1 to check which estimate is closer.

## 5.9 Sensitivity analysis

**Effect of size uncertainty (organic mode):**

| Case | KLOC | Effort (PM) | Time (months) | Staff (persons) | Person-hours |
|---|---|---|---|---|---|
| Low | 3.7 | 9.48 | 5.88 | 1.61 | 1,441 |
| **Nominal** | **5.0** | **13.01** | **6.63** | **1.96** | **1,977** |
| High | 6.4 | 16.85 | 7.31 | 2.30 | 2,562 |

**Effect of project mode (5.0 KLOC), for comparison only:**

| Mode | a | b | c | d | Effort (PM) | Time (months) | Staff |
|---|---|---|---|---|---|---|---|
| **Organic** | **2.4** | **1.05** | **2.5** | **0.38** | **13.01** | **6.63** | **1.96** |
| Semi-detached | 3.0 | 1.12 | 2.5 | 0.35 | 18.20 | 6.90 | 2.64 |
| Embedded | 3.6 | 1.20 | 2.5 | 0.32 | 24.84 | 6.99 | 3.55 |

# 6. Cost Estimation

Two different kinds of cost are estimated separately: **labour cost** derived from COCOMO effort, and **monetary expenses** for external services. They are not added together as if they were the same kind of cost.

## 6.1 Labour cost (from COCOMO)

$$\text{Labour cost} = \text{Effort (person-hours)} \times \text{hourly rate}$$

The student is not paid, so no real salary is assumed. Using a **hypothetical** rate of US$10 per hour purely for illustration:

| Basis | Hours | Hypothetical rate | Illustrative labour cost |
|---|---|---|---|
| COCOMO estimate (organic, 5.0 KLOC) | 1,977 | US$10/h | US$19,768 |
| Agile plan capacity (4 sprints x 10 days x 3 h) | 120 | US$10/h | US$1,200 |

Replace the rate with any other value to recalculate; the formula stays the same.

## 6.2 Monetary expenses (services)

| Item | Basis | Estimated cost (illustrative) | Notes |
|---|---|---|---|
| LLM API | Pay per token; usage-dependent | About US$1-10 for development and demo | Small low-cost model; a few hundred questions; set a spending limit |
| Embedding and reranker models | Open-source, run locally | US$0 | One-time download |
| PostgreSQL and Qdrant | Local Docker containers | US$0 | Managed free tiers available if hosting is needed |
| Frontend hosting (optional) | Free tier (e.g. Vercel Hobby) | US$0 within free-tier limits | Check usage terms before deploying |
| Backend hosting (optional) | Small VM or container host | About US$5-25 per month | Local models need roughly 1-2 GB RAM; free tiers often too small |
| Domain name (optional) | Annual registration | About US$10-15 per year | Not required |

> Monetary amounts are illustrative estimates; provider prices and free-tier limits change and could not be verified at the time of writing. LLM cost = number of questions × (prompt tokens × input price + answer tokens × output price).

## 6.3 Free-tier usage

- Local development needs no paid services except LLM API usage.
- If a public demo is required: free frontend hosting, free managed PostgreSQL and Qdrant tiers, and a paid small backend host for the models.

# 7. Estimation Summary

**5.0 KLOC**
Estimated size (range 3.7-6.4)

**13.01 PM**
Effort (organic)

**6.63 months**
Development time

**1.96**
Average staff (persons)

| Item | Value |
|---|---|
| Estimation model | Basic COCOMO (Boehm, 1981) |
| Project mode | Organic (a = 2.4, b = 1.05, c = 2.5, d = 0.38) |
| Estimated size | 5.0 KLOC delivered application code (range 3.7-6.4); test code about 2.0 KLOC not counted |
| Effort | 13.01 person-months (range 9.5-16.9) |
| Effort in person-hours | About 1,977 hours at 152 h per person-month |
| Development duration | 6.63 months (range 5.9-7.3) |
| Average staffing | 1.96 persons |
| Illustrative labour cost | US$19,768 at a hypothetical US$10/h (formula: hours x rate) |
| Monetary expenses | About US$1-10 LLM API usage (illustrative); optional hosting about US$5-25/month |
| Agile plan (companion document) | 4 sprints of 2 weeks; about 120 h student capacity; scope managed by MoSCoW priority |

# 8. Limitations and Risks

| Topic | Discussion |
|---|---|
| Uncertainty in early KLOC estimates | Size is guessed before design is complete. The 3.7-6.4 KLOC range already changes effort from 9.5 to 16.9 person-months. Re-estimate after Sprint 1 using actual code size. |
| Theoretical estimate vs student schedule | COCOMO predicts about 1,977 hours, while the assignment timetable allows about 120 hours. The model assumes an industrial process and no modern reuse; the student plan relies on frameworks and fixed time boxes. The difference is a real schedule risk, managed by prioritising Must-have scope. |
| Integration and debugging | Connecting the frontend, API, PostgreSQL, Qdrant, local models and the LLM API creates many failure points. Debugging these is hard to size and is a common cause of overruns. |
| Third-party API and hosting dependencies | LLM API availability, rate limits, pricing and model changes are outside the project's control. Hosting free tiers may not provide enough memory for local models. |
| Limits of a code-size model for AI applications | Much of the system's capability comes from pretrained models and libraries (embedding model, cross-encoder, LLM, BM25, vector database). Effort goes into integration, prompt design, retrieval tuning and evaluation, which add few lines of code. KLOC therefore under-represents some work and over-represents other work; Basic COCOMO has no cost drivers to correct for reuse or tools. |

Despite these limits, COCOMO gives a transparent, repeatable baseline. Used together with the Agile plan's sprint-by-sprint tracking, it helps judge early whether scope must be reduced.